

DSL и LSP за генерисање тест података

Лазар Нагулов, SV61/2022

Ментор: проф. др Игор Дејановић

Нови Сад, 2026

Outline

1. Проблем и мотивација
2. Циљ
3. Архитектура система
4. Евалуација
5. Закључак

1. Проблем и мотивација

Ручно / скрипте

```
def generate_user():  
    user = {}  
    user["name"] = random_name()  
    user["age"] = random_age()  
    return user
```

Код + генерација

DSL

```
template User {  
    name = string;  
    age = int [range=1..=100];  
}
```

декларативни опис

Понављање и мешање описа са имплементацијом.

2. Циљ

1. DSL за опис тест података

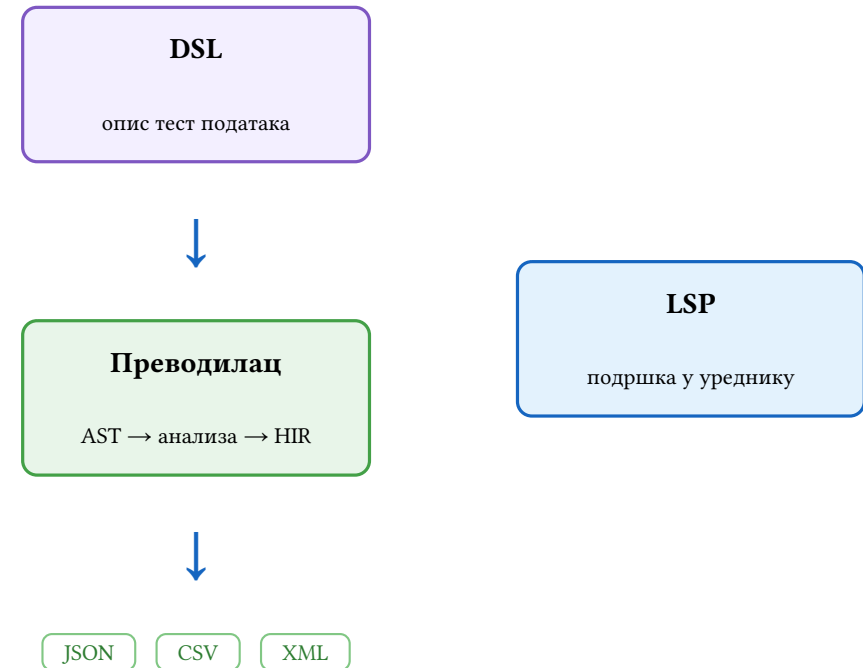
- типови и шаблони
- ограничења и структуре
- референце и енумерације

2. Преводаилац и генерација

- AST, семантичка анализа и HIR
- генерисање JSON, CSV и XML
- систем модула и .tmod

3. LSP подршка

- језички сервер за DSL
- интеграција са уредницима текста
- језичке функционалности током рада



2.1 Пример

DSL

```
@import course;
type StudentId =
  string_pattern "S${#[7]}";

template Student {
  id = StudentId;
  name = string;
  course = ref Course.name;
  year = int [range=1..=4];
}

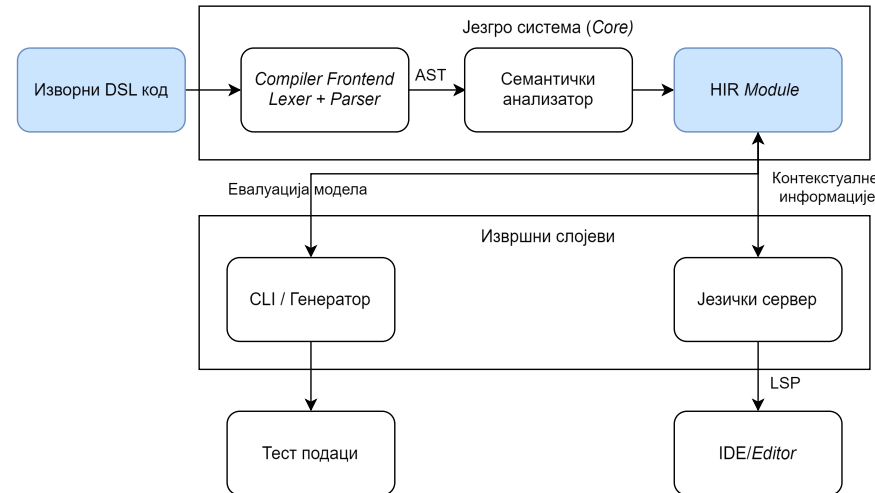
@generate Student[100];
```



Генерисани подаци

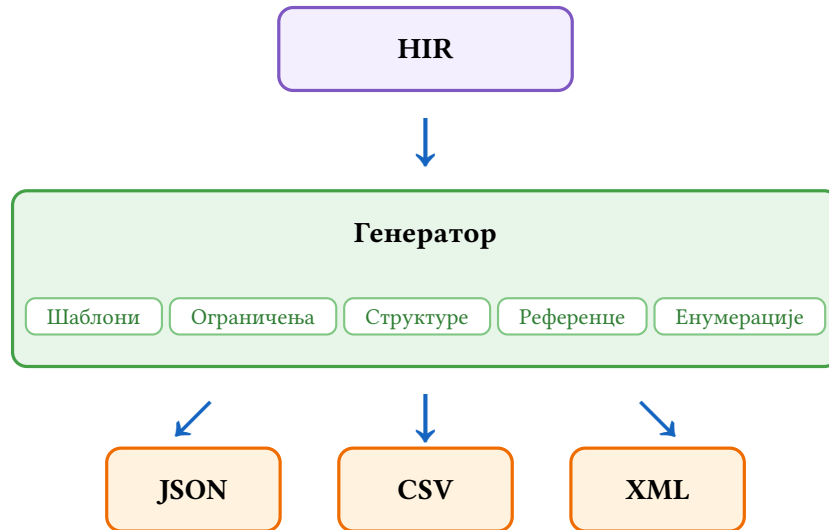
```
[{
  "id": "S4821937",
  "name": "Ana",
  "year": 3,
  "course": "Programming"
},
{
  "id": "S9023421",
  "name": "Marko",
  "year": 2,
  "course": "Web Design"
}...]
```

3. Архитектура система



- **Архитектура дељеног језгра:** Заједнички синтаксни и семантички анализатор за све подсистеме.
- **Раздвајање слојева:** HIR служи као граница између аналитичког језгра и извршних модула (CLI и LSP).

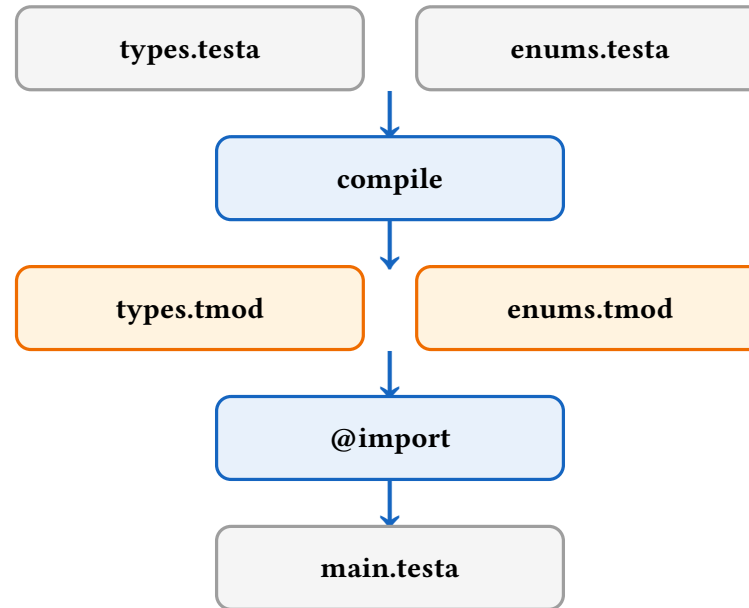
3.1 Архитектура генератора података



```
interface FileGenerator:  
    generate(record) -> String  
    extension() -> String  
    header(fields) -> String?  
    footer() -> String?  
    separator() -> String?
```

- **Јединствен улаз:** HIR служи као заједничка основа за генерисање свих излазних формата.
- **Проширивост излаза:** FileGenerator интерфејс омогућава лако додавање нових формата (JSON, CSV, XML...).

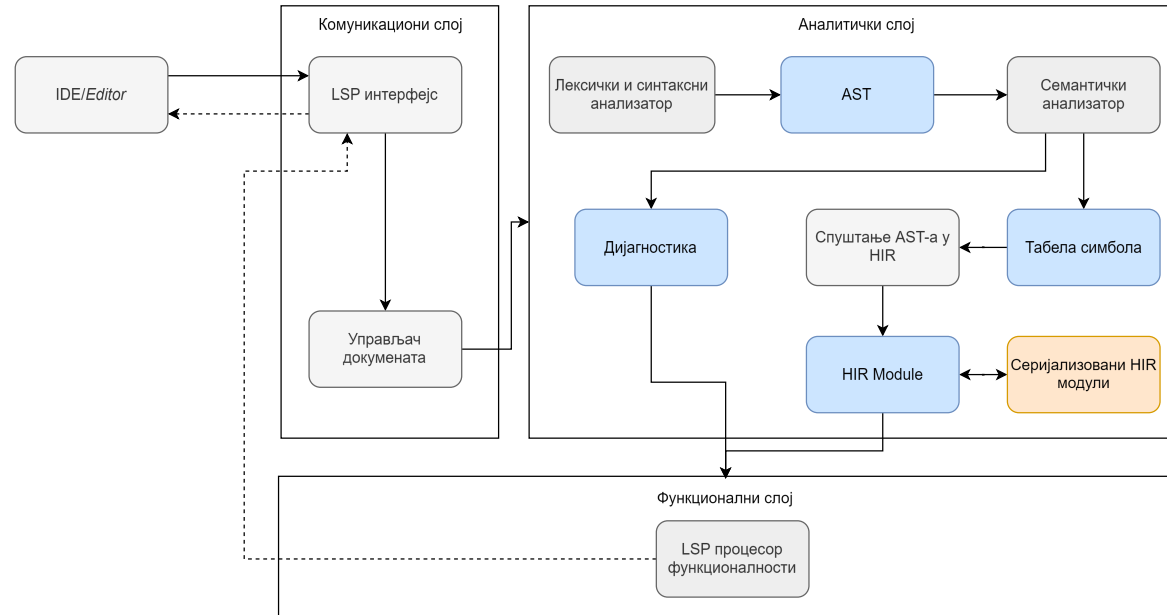
3.2 HIR и систем модула



Независно превођење: Модули се чувају као серијализовани HIR (.tmod).

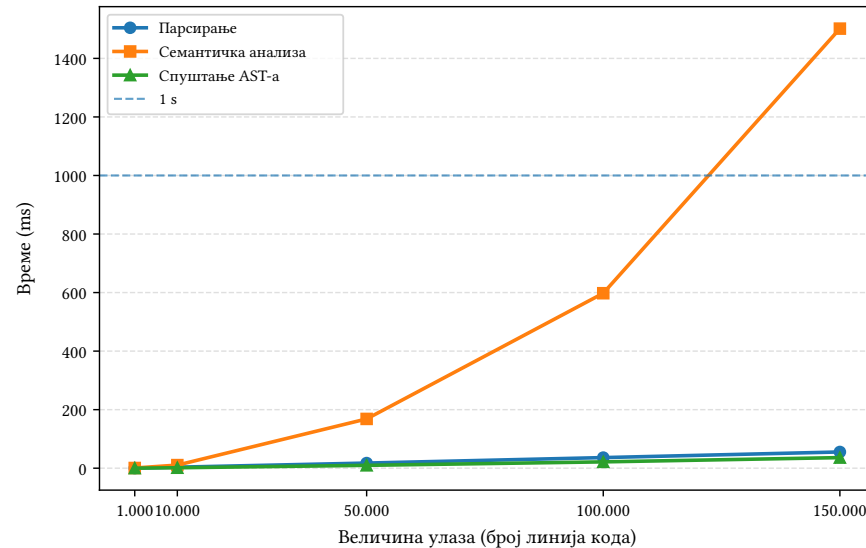
Увоз: Прескочена поновна синтаксна и семантичка анализа.

3.3 Протокол језичких сервера



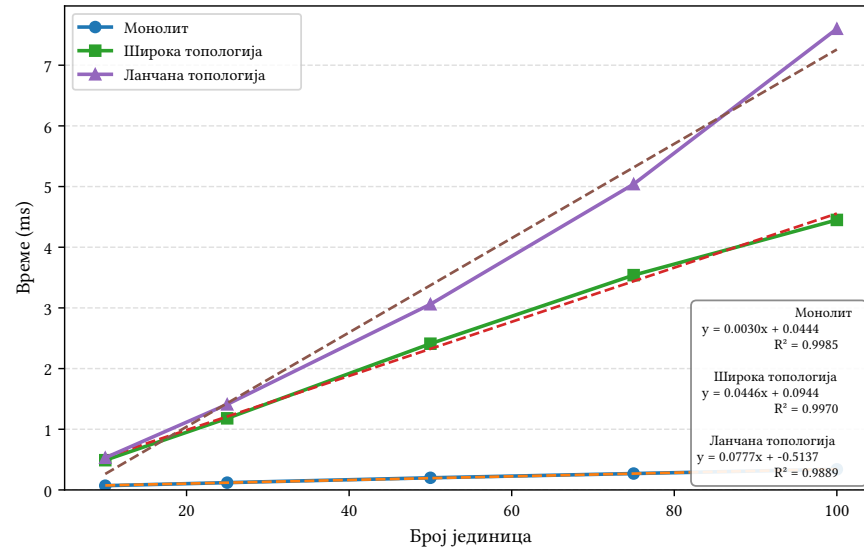
- **Трослојна архитектура:** Раздвојени комуникациони слој, аналитички слој и функционални LSP слој.
- **Инкрементална обрада:** Управљач докумената прати измене отворених докумената.

4.1 Евалуација: Процес превођења



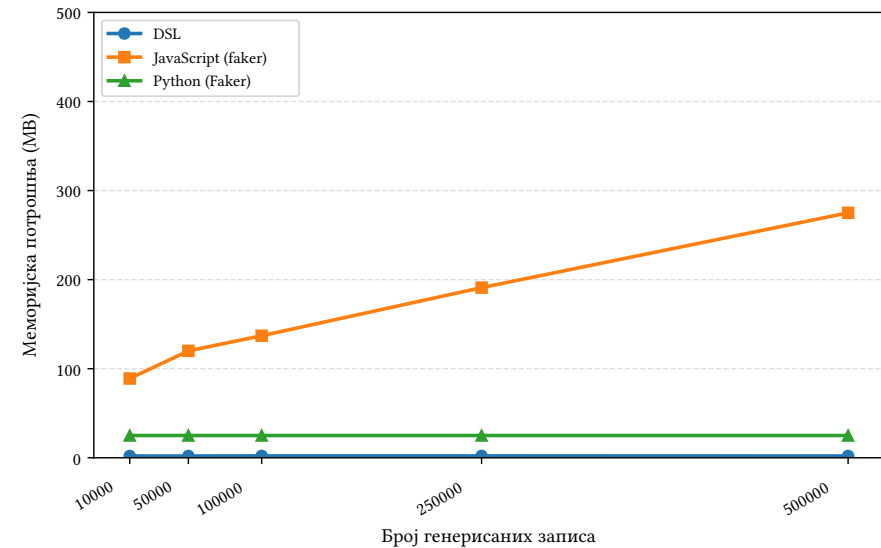
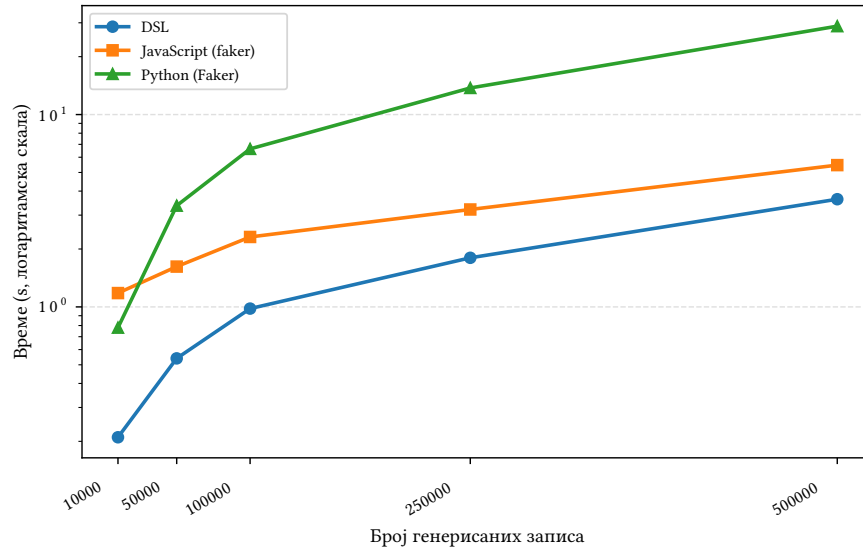
- **Задовољен критеријум интерактивности:** Укупно време превођења је испод прага од 1.000 ms за датотеке до 100.000 линија кода.
- **Уско грло:** Нелинеарни раст фазе семантичке анализе преко 100.000 линија кода указује на потребу за инкременталном анализом.

4.2 Евалуација: Систем модула



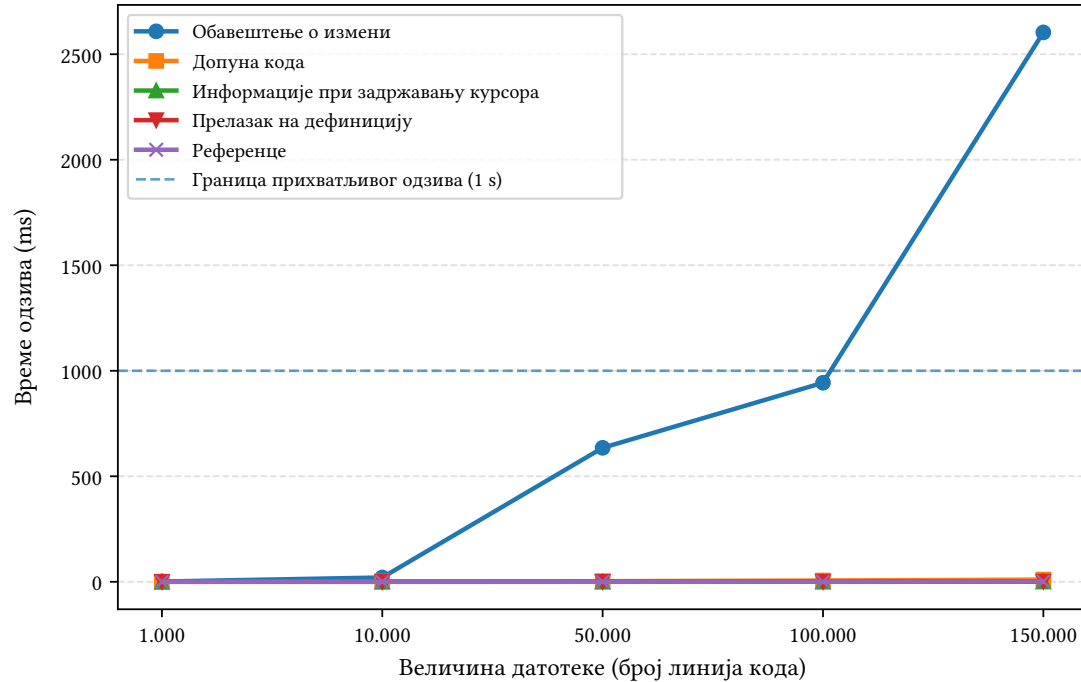
- **Потврђена линеарна сложеност $\mathcal{O}(N)$:** Систем одржава линеарно скалирање времена учитавања независно од топологије стабла зависности.
- **Утицај топологије:** Ланчана топологија остварује највећи трошак услед рекурзивног разрешавања симбола (7,60 ms за 100 модула).

4.3 Евалуација: Генерисање података



- **Временска ефикасност:** Систем остварује боље време извршавања (3,63 s за 500.000 записа) у односу на Node.js (5,46 s) и Python (28,81 s).
- **Меморијска ефикасност $\mathcal{O}(1)$:** Задржана константна потрошња меморије од 2 MB.

4.4 Евалуација: Језички сервер



- **Тренутни одзив (≈ 1 ms):** Операције над табелом симбола раде у константном времену.

5. Закључак

- **Декларативни DSL:** Успешно развијено решење у језику Rust за синтезу података.
- **HIR и модуларност:** Уведена високонивојска међурепрезентација и бинарна серијализација за изоловано превођење.
- **Очување интегритета:** Генератор симулира релационе везе уз одржавање жељених расподела.
- **LSP интеграција:** Реализован језички сервер који обезбеђује статичку анализу унутар развојног окружења.

Будући рад

- **Аутоматизација и инкременталност:** Увођење механизма праћења измена датотека ради аутоматског превођења само измењених модула.
- **Средњенивојска међурепрезентација (MIR):** Декомпозиција конструкта на канонички облик за анализу тока података и елиминацију мртвог кода.
- **Проширење бекенда:** Формирање независног IR слоја који олакшава додавање нових генератора (LLVM IR, C код).

Завршна реч

Хвала на пажњи!

Питања?